

Scalable Real-Time Inference Platform on AWS

Formal Project Document for Institutional Records

Field	Detail
Document Title	Scalable Real-Time Inference Platform on AWS
Document Version	2.0
Date	June 2026
Author / Maintainer	Roshini
Repository	https://github.com/Rosh054/Scalable-Real-Time-Inference-Platform
Classification	Academic / Internship / Professional Portfolio

Document Control and Maintenance Responsibility

This document is the official record of the project. The repository and documentation are maintained under the responsibility of **Roshini**.

When this project is referenced in academic submissions, internship reports, recommendation letters, or correspondence with third parties, the maintaining team must be able to respond to enquiries ...

- This formal document (PDF and markdown source)
- The repository documentation under ``docs/``
- Verifiable artifacts under ``results/``

Updates to project scope, metrics, deployment status, or architecture must be reflected in this document and in ``docs/FORMAL_PROJECT_DOCUMENTATION.md`` when material changes occur.

Academic Project Abstract

Title: Scalable Real-Time Inference Platform on AWS

Abstract:

This project focuses on building a scalable cloud-based machine learning inference platform using AWS ECS, Docker, Redis, PostgreSQL, and CloudWatch. It supports low-latency REST API predictions, c...

Extended summary:

The implementation uses FastAPI for the inference API, Redis for cache-aside response caching, and PostgreSQL for per-request prediction audit logging. Docker Compose provides a validated local env...

Verified performance (local load test): approximately 713 requests per second sustained throughput; 65.9 ms p50, 107.7 ms p95, and 151.3 ms p99 latency; 0% error rate; 99.94% cache hit rate und...

Internship Abstract

****Title:** Cloud-Native Backend and Distributed Systems Internship**

****Abstract:****

This internship focuses on developing scalable backend services using FastAPI, Redis, PostgreSQL, Docker, AWS, and CI/CD workflows. The work involves building REST APIs, asynchronous task processin...

****Implementation alignment (current repository scope):****

Internship theme Delivered in this project

REST APIs FastAPI with Pydantic validation; OpenAPI at /docs
Distributed caching Redis cache-aside with deterministic SHA-256 input hashing
Data persistence PostgreSQL predictions table; SQLAlchemy ORM
Containerisation Dockerfile; Docker Compose (api, postgres, redis)
AWS and cloud deployment Terraform: ECS, ALB, RDS, ElastiCache, S3, CloudWatch
Monitoring dashboards Application /metrics; CloudWatch dashboard and alarms in IaC
CI/CD workflows GitHub Actions: lint, pytest, Docker build, smoke test

****Scope clarification:**** The production codebase implements ****synchronous real-time inference**** (request received and response returned in the same HTTP transaction). Separate asynchronous worker q...

1. Executive Summary

The Scalable Real-Time Inference Platform is an end-to-end machine learning inference service designed for low-latency, auditable, and operationally observable predictions. The system demonstrates ...

****Current validation status:****

- Application stack: ****implemented and tested**** locally via Docker Compose
- Performance: ****documented**** with reproducible load-test artifacts
- AWS infrastructure: ****designed and validated**** in Terraform (static validation and CI)
- Live AWS hosting: ****optional**** (not required for project completion; avoids unnecessary cost)

2. Problem Statement and Objectives

2.1 Problem

Machine learning models are frequently deployed without standard APIs, caching layers, audit trails, or cloud-ready operational patterns. This limits throughput under repeated requests, complicates...

2.2 Objectives

1. Expose a validated REST API for real-time inference
2. Reduce latency for repeated inputs using Redis cache-aside
3. Log every prediction to PostgreSQL for audit and analysis
4. Provide health and metrics endpoints for operations
5. Package the service with Docker for reproducible environments
6. Define AWS production infrastructure in Terraform
7. Automate quality assurance through CI/CD

8. Record reproducible load-test results without fabricated metrics

3. System Architecture

3.1 Target AWS architecture

```
Internet
|
v
Application Load Balancer (public subnets)
|
v
ECS Fargate Service - FastAPI (private subnets)
|   |   |
v   v   v
ElastiCache RDS      S3
Redis  PostgreSQL  model.joblib
|
v
CloudWatch - logs, metrics, alarms
SNS - optional alert notifications

GitHub Actions --> ECR --> ECS deployment
```

3.2 Local validated architecture

```
Client --> FastAPI :8000
|--- Redis (cache)
|--- PostgreSQL (audit log)
v
models/model.joblib
```

3.3 Key design patterns

- **Cache-aside:** Read Redis first; on miss, run inference and store result with TTL
- **Deterministic cache keys:** SHA-256 hash of normalised JSON payload
- **Audit logging:** One database row per prediction with request_id and latency
- **Request tracing:** X-Request-ID header on all HTTP responses
- **Structured logs:** JSON format when APP_ENV=aws for CloudWatch Logs Insights

4. Technology Stack

Layer	Technologies
Language	Python 3.11+
API	FastAPI, Pydantic, Uvicorn
ML	scikit-learn, joblib (Iris RandomForest)
Cache	Redis 7
Database	PostgreSQL 15, SQLAlchemy 2.x, Alembic
Containers	Docker, Docker Compose
Cloud IaC	Terraform - AWS provider 5.x
CI/CD	GitHub Actions
Load testing	hey, Locust

5. API Specification

Method	Path	Description
GET	/health	API, Redis, database, model status; environment and version
POST	/predict	Classification; returns prediction, request_id, cache_hit, latency_ms
GET	/metrics	Prometheus-style counters including cache hit rate
GET	/model-info	Model name, version, input schema, artifact location

Prediction input (Iris features): sepal_length, sepal_width, petal_length, petal_width (float, range 0-30).

6. Database Schema

Table: predictions

Column	Description
id	Primary key
request_id	UUID per request
model_version	Model version string
input_hash	SHA-256 of normalised input
input_payload	JSON request body
prediction	Integer class label
cache_hit	Boolean
latency_ms	Server-side milliseconds
created_at	Timestamp

7. Performance and Test Results

7.1 Load test (local, verified)

Parameter	Value
Date	2026-06-01
Tool	hey
Concurrency	50
Duration	30 seconds
Requests per second	713.18
p50 latency	65.9 ms
p95 latency	107.7 ms
p99 latency	151.3 ms
Error rate	0%
Cache hit rate	99.94%
Uncached latency (approx.)	19.56 ms
Cached latency (approx.)	3.30 ms
Latency reduction from caching	83%

Artifact files: results/metrics_template.md; results/hey_20260601_143951.txt

7.2 Automated tests

Thirteen pytest cases cover health, prediction, validation errors, database logging, cache behaviour, metrics, model loading, request ID middleware, and structured logging. Continuous integration r...

8. Deployment Status

Environment	Status	Notes
-------------	--------	-------

Docker Compose (local)	Complete	make up; make verify
------------------------	----------	----------------------

GitHub Actions CI	Complete	ci.yml, terraform.yml
-------------------	----------	-----------------------

AWS (live)	IaC ready	Optional terraform apply; incurs cost
------------	-----------	---------------------------------------

****Official statement for correspondence:**** The project implements and validates the application locally and designs the AWS production stack in Terraform. Claims of live AWS operation require a com...

9. Verification for Third Parties

```
git clone https://github.com/Rosh054/Scalable-Real-Time-Inference-Platform.git
cd Scalable-Real-Time-Inference-Platform
make setup
source .venv/bin/activate
make train-model
make up
make verify
```

Detailed instructions: docs/VERIFICATION.md in the repository.

10. CI/CD Pipelines

Workflow	Purpose
----------	---------

ci.yml	Ruff lint, pytest, Docker build, Compose smoke test
--------	---

terraform.yml	terraform fmt and validate
---------------	----------------------------

deploy.yml	Optional ECR push and ECS deploy (requires AWS secrets)
------------	---

11. AWS Services (Terraform)

AWS Service	Role
-------------	------

ECS Fargate	Run containerised FastAPI service
-------------	-----------------------------------

Application Load Balancer	HTTP routing and health checks
---------------------------	--------------------------------

ECR	Container image registry
-----	--------------------------

RDS PostgreSQL	Prediction audit database
----------------	---------------------------

ElastiCache Redis	Response cache
-------------------	----------------

S3	Model artifact storage
----	------------------------

CloudWatch Logs, dashboard, operational alarms
IAM Task and execution roles
VPC Public and private subnets with NAT

12. Future Enhancements

- Asynchronous inference queues (SQS, worker services)
- AWS Secrets Manager for database credentials
- HTTPS on ALB with ACM certificates
- Multi-AZ RDS and Redis replication
- Batch inference endpoints

13. Repository Documentation Index

Document Purpose

docs/FORMAL_PROJECT_DOCUMENTATION.md Complete formal record
docs/DOCUMENTATION_INDEX.md Documentation catalogue
docs/LETTER_SUPPORT_SUMMARY.md One-page letter support
docs/VERIFICATION.md Reproduction guide
docs/AWS_ARCHITECTURE.md AWS design detail
README.md Repository overview

14. Enquiries and Contact

Technical and institutional enquiries regarding this project should be directed to the repository maintainer, ****Roshini****, with reference to this document and the GitHub repository:

<https://github.com/Rosh054/Scalable-Real-Time-Inference-Platform>

15. Approval Record

Role Name Date Signature

Project author / maintainer Roshini June 2026

Technical reviewer

Institutional approver

End of formal project document.